Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

CI-4835 Redes de Computadoras I



Chat multiusuario sobre TCP

Estudiantes:

Nazareth Colmenares. 20-10017

Gaston García. 19-10668

Amanda Vargas. 20-10081

Profesor:

Samuel Sanchez

Chat multiusuario sobre TCP

El proyecto consiste en un sistema de chat multiusuario orientado a texto, desarrollado en C++17 mediante la API de sockets POSIX. Su propósito es integrar la comunicación entre usuarios, el almacenamiento de mensajes privados para destinatarios desconectados y la consulta del estado del servicio mediante HTTP. Para ello, se dispone de un cliente de terminal y de un servidor que reúne los tres servicios en un mismo proceso. La implementación utiliza estructuras de la biblioteca estándar y realiza directamente el manejo de la red y del protocolo.

El servidor utiliza un único ciclo poll() con sockets no bloqueantes. Se eligió para atender conexiones sin crear un hilo por usuario y simplificar el manejo del estado compartido. Cada cliente dispone de una cola de salida y un desplazamiento para completar envíos parciales; si temporalmente no puede enviar, el servidor continúa con los demás. La cola se limita a 1 MiB para controlar clientes que dejan de leer.

La delimitación se realiza manualmente: los bytes recibidos se acumulan hasta encontrar CRLF, admitiendo comandos fragmentados o agrupados. Las líneas del cliente tienen un máximo de 512 bytes, incluido el terminador. Ante un exceso, se responde 502 y se descarta hasta el siguiente CRLF, sin almacenar la entrada completa. También se validan UTF-8 y argumentos. El cliente vigila teclado y socket con poll para mostrar eventos asíncronos.

Los privados para usuarios conocidos y desconectados se guardan en archivos .araguaney, con cabeceras, cuerpo y terminaciones CRLF. Los buzones sobreviven al reinicio. La numeración permanece estable durante la sesión: DELE solo marca y QUIT confirma el borrado; una desconexión sin QUIT conserva los archivos. El historial de cada canal mantiene sus últimas 50 publicaciones en memoria.

La puerta HTTP comparte el proceso servidor y lee hasta CRLF. Atiende GET /usuarios, /canales e /historial/, con códigos 200, 400, 404 y 405. Incluye las cabeceras exigidas, calcula Content-Length en bytes y cierra después de enviar la respuesta.

Durante el desarrollo se utilizó OpenAI, para proponer y generar bloques de código, incluidos concurrencia, delimitación, validadores, buzones, HTTP y adaptación del cliente.